In this manual

1. Overview and included content
2. Installation and first successful play
3. Folder structure and source reference
4. Ball Breaker Physics reskin: interface colours and visible text
5. Gameplay and content
6. Boosters
7. Manual editor and generation
8. LiveOps, ads and haptics
9. Development controls
10. Android release
11. WebGL
12. Troubleshooting
13. Artwork, fonts and audio replacement
14. Level editor: concrete editing workflow
15. Procedural generation and level data reference
16. Authoring controls and booster configuration
17. Game text versus saved data and service configuration
18. Reskin troubleshooting and release check
19. Licences, support and documentation access

Ball Breaker Physics

Developer manual · Version 1.0.0 · SatisVizion · Documentation revised 26 October 2026

Overview and included content

Drag down from the launcher to aim and release a volley of balls. Reduce numbered shapes to zero. Shapes rise after each volley; clear them before they reach the top danger line.

This is an editable Unity game template and starting point for your own title. Customize the visuals, copy, level progression and release settings for your audience, then test your changes. The included Android demo is a showcase build; it is not a signing key or a production store submission.

Deterministic fixed-step ball simulation, numbered neon shapes, rising rows, aiming guidance, volley budgets and authored difficulty progression. The campaign contains 25 authored levels and game-specific procedural continuation. The included service kit is HyperCasual LiveOps Kit. Ads ship in Dummy/simulated mode and IAP is Disabled; third-party advertising/billing SDKs are not bundled.

Installation and first successful play

1. Back up your project and extract the main ZIP completely. Read `Documentation/Documentation.html`

before importing the package. Offline documents do not require the hosted page to be available.

2. Install Unity `6000.0.62f1` through Unity Hub. Add Android Build Support (including the matching SDK, NDK and OpenJDK) if building Android, and WebGL Build Support if building browser demos. A supplied source project can be opened with Add project in Unity Hub.
3. For package import into a new project, select a Universal Render Pipeline template. Use `Assets > Import Package > Custom Package` and import the complete package. Do not omit the private game Kit, HyperCasualLiveOpsKit, fonts, settings, scenes or their .meta files.
4. Check `Window > Package Manager`: Universal RP `17.0.4`, Unity UI `2.0.0` and Input System `1.14.2`. Standard Unity modules include `com.unity.modules.screencapture` and `com.unity.modules.androidjni`. Let Unity finish compilation before opening a scene.
5. Set `Edit > Project Settings > Player > Other Settings > Active Input Handling` to `Both` manually. Restart if Unity requests it. The documented setup is a user action; do not rely on an import hook changing a buyer's Project Settings.
6. In `Edit > Project Settings > Graphics` select the supplied URP asset, and check the pipeline assignment for the active Quality level. Retain the packaged Settings assets and GUID references. A pink scene usually indicates a missing pipeline or shader assignment.
7. Open `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scenes/BallBreaker_Mobile.unity`. Press Play, focus the Game view, and follow the first-level tutorial: Drag down from the launcher to aim and release a volley of balls. Reduce numbered shapes to zero. Shapes rise after each volley; clear them before they reach the top danger line.
8. Stop Play mode before editing source or authored content. Open the landscape scene `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scenes/BallBreaker_WebGL.unity` and verify the alternate layout as well. Runtime-created UI changes made only in Play mode are not a saved reskin.

Folder structure and source reference

The concrete paths in this manual match this Unity Asset Store single-product-root layout. The game root is `Assets/SatisVizionBallBreaker/Games/BallBreaker`. Keep .meta files with their assets so scene, font, sprite and material references remain valid.

```
Assets/SatisVizionBallBreaker/Games/BallBreaker/
  Scenes/            Mobile and landscape entry scenes
  Scripts/           Rules, HUD, board view and game data
  Editor/            Game-specific authoring tools
  Resources/         Level JSON, artwork and game audio
  Kit/               Required game runtime and editor helpers
  Documentation/     This manual
Assets/SatisVizionBallBreaker/HyperCasualLiveOpsKit/        Included service kit
Assets/SatisVizionBallBreaker/TextMesh Pro/              Required text resources
```

Exact file	What to edit or inspect
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BallBreakerGame.cs`	Main game lifecycle, progression, input/rule orchestration and booster handlers.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BallBreakerGame.Hud.cs`	Runtime HUD, menu and popup construction, display strings and tutorial presentation.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BallBreakerGame.View.cs`	Scene geometry or board rendering, camera layout and visual feedback.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbArt.cs`	Colour palette, resource artwork loading and generated visual helpers.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbBot.cs`	Authoring assistance and automated move/validation logic.

`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbData.cs`	Serializable level and gameplay data structures.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbGen.cs`	Procedural level creation and seed/difficulty rules.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbLevels.cs`	Campaign data loading, level access and procedural continuation.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbSim.cs`	Game-specific structures and helpers; inspect the public types and call sites before extending them.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Kit/Runtime/HzUI.cs`	Creates the uGUI labels/buttons and resolves the game kit fonts.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Kit/Runtime/HzAudio.cs`	Named sound resources, music playback where provided and fallback sound synthesis.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Kit/Runtime/HzCore.cs`	Game-prefixed local persistence helpers.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Kit/Runtime/HzCore.cs`	Bridge from game calls to simulated/service-kit ads and native haptic hooks.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Editor/BallBreakerLevelEditor.cs`	The visual level editor and its actual menu commands.

Ball Breaker Physics reskin: interface colours and visible text

The interface is constructed in C# when the scene runs. There is no single static scene prefab containing every final label. Use the exact game files below; changing only a shared-kit theme or localization table will not replace game-authored HUD/menu strings.

1. Change interface colours

Open `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbArt.cs` in your C# editor. Find these declared palette symbols; change only the quoted hexadecimal value in the existing `HzUI.Hex(...)` assignment. Preserve the symbol, type and surrounding syntax. This keeps every caller using the shared colour in sync.

File / symbol	Current value	Concrete usage to inspect
`Scripts/BbArt.cs BgTop`	`#1E1B44`	`BallBreakerGame.View.cs: var grad = HzUI.Img("Grad", _bgLayer, HzUI.Gradient(BgTop, BgBottom), Color.white);`
`Scripts/BbArt.cs BgBottom`	`#0E1028`	`BallBreakerGame.View.cs: var grad = HzUI.Img("Grad", _bgLayer, HzUI.Gradient(BgTop, BgBottom), Color.white);`
`Scripts/BbArt.cs Panel`	`#23224F`	`BallBreakerGame.Hud.cs: var set = HzIcons.IconButton(_column, HzIcons.Settings, Panel, () => { HzAudio.Play(BbSfx.Click, 0.6f); ShowSettingsPopup(); }, 116);`
`Scripts/BbArt.cs Ink`	`#141632`	`BallBreakerGame.Hud.cs: v.BadgeText.color = _guide == b || c <= 0 ? Ink : Color.white;`
`Scripts/BbArt.cs Mint`	`#3DF5B0`	`BallBreakerGame.Hud.cs: v.Badge.GetComponent<Image>().color = _guide == b || c <= 0 ? Mint : Coral;`
`Scripts/BbArt.cs Cyan`	`#3CC8FF`	`BallBreakerGame.Hud.cs: _nameTxt = HzUI.Label("Name", lvl.transform, "", 32, Cyan);`

`Scripts/BbArt.cs Coral`	`#FF5A5F`	`BallBreakerGame.Hud.cs: var bd = HzUI.Panel("Badge", v.Root, Coral, 30); bd.raycastTarget = false;`
`Scripts/BbArt.cs PopupInk`	`#2A2A5A`	Palette helper in `BbArt.cs`; search `PopupInk` in its call sites, including generated visuals.

Concrete first colour edit: in `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbArt.cs`, find this fragment within its existing declaration:

```
Panel = HzUI.Hex("#23224F")
```

Replace only that colour value, for example:

```
Panel = HzUI.Hex("#24485C")
```

1. Save the source and return to Unity. Wait for compilation, resolve any Console errors, and start a fresh Play session so the UI and cached generated artwork are rebuilt.
2. Inspect the changed HUD surface, settings menu and a win/lose or booster popup. Verify that text remains readable, selected/disabled states remain distinct, and icons are visible against the new background.
3. Check the mobile and landscape scenes. Some popup, world, camera and material colours are separate literal assignments in `Scripts/BallBreakerGame.Hud.cs` or the .View.cs file. If one surface did not change, inspect the colour argument used by that exact control instead of replacing every matching hex value globally.
4. For a procedural colour change, stop and re-enter Play; if you deliberately disabled domain reload, restart the editor when testing static sprite/material caches. A bitmap with baked-in colours must be edited separately from the C# palette.

2. Change a visible HUD label

Open `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BallBreakerGame.Hud.cs`, find `RefreshHud` and locate this exact source line:

```
_levelTxt.text = "LEVEL " + _playLevel.ToString(CultureInfo.InvariantCulture);
```

To call levels stages, change the display string while preserving the level expression and formatting:

```
_levelTxt.text = "Stage " + _playLevel.ToString(CultureInfo.InvariantCulture);
```

This is a targeted edit to the displayed string, not a global search/replace of object names. `HzUI.Label("Lvl", parent, "LEVEL " + Level, ...)` uses its first string as a hierarchy/object name and its third argument as visible text. In assignments such as `_levelTxt.text = ...`, edit only the literal part; retain the number, interpolation braces, arithmetic, conditional branches and formatting.

3. Edit menus, popups, title and booster captions

Use this game-specific lookup table. The snippet is a search target in the named file; it shows the actual display-writing call rather than an assumed localization key. Line numbers are navigation hints for this revision and can shift after your edits.

Exact file and method	Search for this source

Scripts/BallBreakerGame.Hud.cs RefreshHud line 148	`_levelTxt.text = "LEVEL " + _playLevel.ToString(CultureInfo.InvariantCulture);`
Scripts/BallBreakerGame.Hud.cs ShowSettingsPopup line 476	`var p = BbPopup("Settings", "", Cyan, null, Color.white, null, -1,`
Scripts/BallBreakerGame.Hud.cs HelpPage line 509	`BbPopup("How to Play " + (i + 1) + "/" + HelpPages.Length, HelpPages[i], Cyan, art, c, null, -1,`
Scripts/BallBreakerGame.cs class declarations line 21	`public override string Title => "Ball Breaker Physics";`
Scripts/BallBreakerGame.cs Win line 341	`BbPopup(_stars == 3 ? "Perfect!" : "Level Clear!", msg, Mint, Get("win_badge"), Color.white, D.name, _stars,`
Scripts/BallBreakerGame.cs Lose line 361	`BbPopup("Over the Line!", "A shape crossed the dashed line.\nClear the top rows and keep going?", Coral, null, Color.white, null, -1,`
Scripts/BallBreakerGame.cs GameOver line 378	`BbPopup("Try Again!", "Aim for the shapes closest to the line\nand grab every + ring you can.", Coral, null, Color.white, D.name, -1, ("Retry", Mint, StartLevel));`
Scripts/BallBreakerGame.Hud.cs BuildSides line 133	`var t = HzUI.Label("Title", side, "HOW TO PLAY", 50, Cyan); t.rectTransform.Place(new Vector2(0.5f, 1), new Vector2(w - 40, 80), new Vector2(0, -40), new Vector2(0.5f, 1));`

1. Change the `Title` property in `Scripts/BallBreakerGame.cs` for the game title exposed by the base game. Also change any separate literal title built by the HUD/view; a Title property is not a universal replacement for all title art.
2. Change visible menu captions in `ShowSettingsPopup` and the help-page strings in `BallBreakerGame.Hud.cs`. Keep action callbacks, toggle variables and control object names unchanged.
3. For booster names edit `BoostName` in `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BallBreakerGame.cs`. For booster help edit `BoostDesc` in `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BallBreakerGame.Hud.cs`. Preserve array lengths/order, booster IDs, unlock logic and charge values. A cosmetic caption must not rename an ad placement, resource lookup or PlayerPrefs key.
4. For status counters, change the actual refresh method, not only the label initializer; otherwise the next frame or level load may overwrite your new caption. Update singular/plural and portrait/landscape variants together.
5. Translate tutorial/help/result text in the game files and level `name` / `hint` fields where used. Keep `\n` line breaks and `{...}` interpolations valid, and test long lines in narrow portrait popups.

4. What the shared localization guide does and does not cover

The shared HyperCasual LiveOps Kit localization guide concerns UI and content that use that kit's localization service. This game's literal strings in the files above are not automatically replaced by adding entries to an HLK table. For one-language reskinning, edit the literals and data shown here. For runtime language selection,

explicitly route those game display assignments through your chosen lookup service, provide keys for each caption, and refresh active labels when the language changes. That is an integration task, not a delivered translation toggle.

BallBreakerGame.Hud.cs writes LEVEL, VOLLEY, BALLS x and PAR counters and the HOW TO PLAY side panel. BallBreakerGame.cs contains Over the Line!, Try Again! and result text. Change only the display words around the numbers. The shape hit-point data, volley state and dashed-line failure rules remain unchanged by a reskin. BbArt separates field/background colours from panel and popup text colours.

5. Verify the reskin

1. Launch both supplied scenes with a normal player save. Confirm the edited level heading, game title, settings/help buttons, counters and booster names.
2. Open a result popup and a refill offer, and replay the first-level and first-booster tutorials using a disposable test profile or the documented developer controls. Confirm that the changed colours remain readable behind all overlays.
3. Test one valid move, one invalid action, progression to the next level, a retry and one available booster. A text reskin must not change goals, costs, rule checks or resource IDs.
4. Make a new Android/WebGL test build after source edits. Existing hosted demos and videos will continue to show their previously built appearance until rebuilt; editing this manual alone does not change those binaries.

Gameplay and content

Drag down from the launcher to aim and release a volley of balls. Reduce numbered shapes to zero. Shapes rise after each volley; clear them before they reach the top danger line.

Deterministic fixed-step ball simulation, numbered neon shapes, rising rows, aiming guidance, volley budgets and authored difficulty progression.

The source includes **25 authored levels** and seeded procedural continuation. Progress, settings and charges are stored locally with PlayerPrefs. Browser storage depends on the browser profile. The template includes source and editable level data; generated continuation is not advertised as hundreds of individually authored levels.

Boosters

- **Zap** unlocks at level **4**.
- **Push Back** unlocks at level **7**.
- **Ball Burst** unlocks at level **11**.
- **Quake** unlocks at level **16**.

First-use guidance introduces each booster. Availability depends on the level, current state and remaining charges. Rewarded placements are simulated by default; only a completed reward callback should grant a refill. Use the DEV panel to unlock boosters for authoring, not as a substitute for real progression testing.

Manual editor and generation

Open **Tools > Ball Breaker Physics > Level Editor**. Duplicate a level before changing the original tutorial. The authoring file is `Assets/SatisVizionBallBreaker/Games/BallBreaker/Resources/ballbreaker/Levels/levels.json`.

The visual editor places numbered shapes and physics pieces on the board, and configures the level name, hints, ball/volley settings and progression. Preserve the launcher area and top danger line. The same fixed-step ball simulation drives the aim guide, bot, generator and authoring checks. For a first edit, select a non-tutorial level, Dup, rename the copy and reduce one numbered shape hit count within the positive range shown in the inspector. Validate the changed board, inspect the result, Save, then Play this level. Avoid placing an obstacle across the launcher or filling every escape path. Re-run Validate after moving a shape. Generation applies its seed and difficulty settings, then checks the simulated solving result before runtime delivery.

The included editor supports saving, duplication and ordering, and provides generation and validation controls appropriate for this game. Read the actual validation result; a difficulty target is not a guarantee of player success. Use a repeatable seed when comparing generated variants. Test changed boards in Play mode.

First authored level schema


```
{
  "levels": [
    {
      "name": "First Drop",
      "hint": "",
      "balls": 3,
      "par": 2,
      "breather": false,
      "layout": 0,
      "diff": 18.7,
      "pieces": [
        {
          "kind": 0,
          "x": 0.53878415,
          "row": 3,
          "rot": 35,
          "hp": 2
        },
        {
          "kind": 0,
          "x": 4.5313416,
          "row": 3,
          "rot": 20,
          "hp": 1
        },
        {
          "kind": 0,
          "x": 6.5,
          "row": 3,
          "rot": 55,
          "hp": 1
        },
        {
          "kind": 0,
          "x": 1.9727069,
          "row": 2,
          "rot": 20,
          "hp": 2
        },
        {
          "kind": 0,
          "x": 3.0474348,
          "row": 2,
          "rot": 75,
          "hp": 2
        },
        {
          "kind": 0,
          "x": 3.9824421,
          "row": 2,
          "rot": 60,
          "hp": 2
        },
        {
          "kind": 0,
```

```
            "x": 6.031328,
            "row": 2,
            "rot": 60,
            "hp": 1
          }
        ]
      }
    ]
  }
```

LiveOps, ads and haptics

The publisher-owned **HyperCasual LiveOps Kit** is included. Read its complete offline manual.

Settings: `Assets/SatisVizionBallBreaker/HyperCasualLiveOpsKit/Resources/HLK/HLKSettings.asset`. Shipping defaults are **Ads Mode Dummy (0)** and **IAP Mode Disabled (3)**. Banner, interstitial and rewarded placements are simulations. **No Google Mobile Ads, billing or other vendor SDK is included.** Automatic vendor-SDK installers are disabled. The game's own Kit is required code, not an advertising SDK.

To enable a real provider, separately install its official SDK, follow the kit manual, configure your own identifiers, consent and privacy policy, then test the selected platform and all cancellation/failure cases. Do not enable SDK symbols without the corresponding dependency. Optional kit modules are capabilities, not production services already configured for this game. Native vibration needs a supported device; browser haptics and other native services have platform limits.

Development controls

Open DEV, press F2/backquote, or use the kit's top-left multi-tap shortcut. The global panel supports level jumps, booster access, ads and haptics controls. Add `HZ_RELEASE` to scripting defines when you want these development controls excluded from a consumer build. This does not configure real ads or certify store readiness.

Android release

Use the Mobile scene, portrait orientation and ARM64/IL2CPP with matching Unity tools. The provided APK is a **debug-signed development demo**, not a production store release. For release turn off Development Build and debugging, choose your own application identifier and review current store API requirements.

In Player → Publishing Settings create or select **your own keystore and alias**. Securely retain the keystore, alias and passwords for future updates; never redistribute signing secrets. Enable Build App Bundle for a Google Play AAB. Install a release candidate on a physical Android device and check touch, audio, haptics, pause/resume, saves and any enabled provider flows. Complete destination-store ads/data declarations and your privacy policy for the services actually enabled.

WebGL

Build the Mobile scene for a portrait-only demo with the built-in Default template and Brotli decompression fallback. Use **Tools → Ball Breaker Physics → Create Responsive WebGL Page** after building and select the output folder. This generates an original responsive page around the complete Build folder. Serve every loader, framework, data and WASM file over HTTP(S); `file://` is unsupported. Audio normally starts after a first click/tap.

The two-scene landscape-first route supports `?mobile=1` to switch to the following Mobile scene. The included WebGL build menu uses HLK's non-development **local showcase** scope. It supports simulated demos;

configure your production providers and resolve the kit build-doctor checks for a real service release.

Troubleshooting

- Missing types: install the required packages and import all three roots, preserving `.meta` files.
- Pointer input exception: select Active Input Handling Both and restart Unity.
- Pink materials: use URP 17.0.4 and run the game's Setup command.
- Sparse scene in edit mode: world/UI objects are built at runtime; press Play.
- No real ad revenue: Dummy mode is simulated and no provider SDK is bundled.
- IAP is unavailable: it is deliberately Disabled by default; configure an official billing integration separately if you need it.
- Progress persists: use the development reset controls or deliberately clear this product's PlayerPrefs.
- WebGL does not load from a local file: serve the complete output over HTTP(S).

Artwork, fonts and audio replacement

Game-specific resource directories in this edition:

Path	Purpose
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Resources/ballbreaker/Art`	Art assets loaded by the game; keep resource base names stable unless you update every lookup.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Resources/ballbreaker/Levels`	Levels assets loaded by the game; keep resource base names stable unless you update every lookup.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Resources/ballbreaker/Music`	Music assets loaded by the game; keep resource base names stable unless you update every lookup.
`Assets/SatisVizionBallBreaker/Games/BallBreaker/Resources/ballbreaker/Sfx`	Sfx assets loaded by the game; keep resource base names stable unless you update every lookup.

1. Replace an existing sprite or audio clip through Unity's Project view while keeping its resource base name and .meta/GUID. If you rename or move it, update its `Resources.Load` lookup and any serialized references together.
2. For a UI icon, find the `Art(...)` / art-class `Get(...)` call in the HUD script and its base name under the Art directory. Imported sprite colours may be baked into the pixels; palette edits only affect code-applied tint or generated geometry.
3. For a 3D scene, inspect the `.View.cs` file and any game-specific material/import helper before rerunning material generation. Back up custom materials first; generation commands may overwrite generated material settings.
4. The game kit resolves its fonts in `Assets/SatisVizionBallBreaker/Games/BallBreaker/Kit/Runtime/HzUI.cs`. Read `Display` and `Body` for the current resource paths. Use fonts with redistribution rights, retain their licence notices, and test accented characters/glyph coverage at the existing sizes.
5. For sound effects, inspect the game's sound-name constants/calls and `Assets/SatisVizionBallBreaker/Games/BallBreaker/Kit/Runtime/HzAudio.cs`. Replace the matching Sfx resource without changing its base name. Named event variants and synthesized fallbacks are handled by the kit; a new filename does nothing unless a caller requests it.
6. Music is a separate resource/playback path. Match the `PlayMusic` lookup if this game supplies a track. Set reasonable clip/volume settings and test mute, focus/background behaviour and device output. Do not infer audio-device testing from a successful build.

Retain the original licence/attribution files for fonts, icons, audio and other included dependencies when distributing your customized game.

Level editor: concrete editing workflow

Open `Tools > Ball Breaker Physics > Level Editor`. Its implementation is `Assets/SatisVizionBallBreaker/Games/BallBreaker/Editor/BallBreakerLevelEditor.cs`. The following names are taken from that editor, so use the displayed control name rather than assuming a generic inspector.

Control / field	Practical use
`Undo`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Reload`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Validate`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Validate All`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Play this level`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Add`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Dup`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Del`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Up`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Down`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Level`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Name`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`First-time hint`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Breather`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Pieces`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Numbers x0.9`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Numbers x1.1`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`All rows up`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`All rows down`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.

`Selected:`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`X`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.
`Rotation`	Select the relevant level first; read the adjacent controls/help in the editor before applying it. Save only after validating the changed content.

1. Exit Play mode. Back up `Assets/SatisVizionBallBreaker/Games/BallBreaker/Resources/ballbreaker/Levels/levels.json`. Select an authored non-tutorial level to avoid unintentionally rewriting the onboarding sequence.
2. Use the available duplication/add control to make a working copy when offered. Change its display Name (or Picture name) and one gameplay property using the visual editor. Keep the first tutorial level intact until you understand its scripted guidance.
3. Run the editor's validation control and read the concrete warnings/errors. Fix missing references, invalid board geometry, impossible goals or malformed data before saving.
4. Save using the editor's Save control. If you intentionally edit JSON externally, save valid JSON, then use Reload/Reload from disk in the editor before continuing. Do not keep two unsaved versions of the same data open.
5. Use Play this level when available, or the documented DEV level navigation, to play the exact changed level. A solver/bot result is useful authoring evidence but does not replace checking the human controls, timing, tutorial and reward flow.

Procedural generation and level data reference

The level data used by this game is located here: `Assets/SatisVizionBallBreaker/Games/BallBreaker/Resources/ballbreaker/Levels/levels.json`. Generation logic is in `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BbGen.cs`. Keep the authored campaign and generated continuation distinct when estimating your content count.

Generation controls exposed by the game editor include: `Breather`, `Auto Generate`, `Seed`.

1. Back up the level JSON before a generation operation. Choose a small trial range and a repeatable seed first, then inspect what is appended or replaced. Do not assume a command is append-only; a selected-level regeneration or Rebuild command can replace authored content.
2. Read the game-specific fields in the level editor and generator source. Match generated board dimensions, rule limits and special mechanics to the intended difficulty; do not copy parameters from another game.
3. Validate generated output, play several early/mid/late boards, and check that the tutorial still introduces mechanics before the generated content requires them. Keep a saved known-good seed for regressions.
4. Treat visual difficulty labels and bot/solver scores as authoring aids. They do not guarantee that every generated board is suitable for every player or that a target completion time is achievable.

Authored field	Type / value in the included sample
`name`	`"First Drop"`
`hint`	`""`
`balls`	`3`
`par`	`2`
`breather`	`false`

`layout`	`0`
`diff`	`18.7`
`pieces`	`array of 7 entries`

Use these field names exactly. JSON keys are serialized identifiers, not interface captions; changing a field name requires corresponding changes to the serializable model, loader, editor and generator. Duplicate levels through the editor when possible rather than guessing a nested schema.

Authoring controls and booster configuration

Actual control	Meaning
`Undo`	Revert the last supported editor operation.
`Reload`	Reload saved JSON from disk; unsaved edits can be discarded.
`Validate`	Check the selected level and read the specific diagnostic.
`Validate All`	Check the whole campaign and inspect every failure.
`Play this level`	Launch the selected board for actual input/play testing.
`Add`	Create a new entry, configure it and validate before saving.
`Dup`	Duplicate the selected level before changing a working copy.
`Del`	Delete the selected entry; back up campaign data first.
`Up`	Move the selected level earlier in the campaign.
`Down`	Move it later in the campaign.
`Name`	Player-facing level name; do not rename the serialized field.
`First-time hint`	Authored hint text; keep the instructions consistent with gameplay.
`Seed`	A repeatable random seed for reproducing a generated result.
`Start difficulty`	Difficulty target at the beginning of the range.
`End difficulty`	Difficulty target at its end.
`Replace from selection`	Can overwrite entries starting at the current selection; back up first.

Effect, unlock and first grant

The descriptions below are the actual game-authored booster help. Unlock and starting inventory are read from `Assets/SatisVizionBallBreaker/Games/BallBreaker/Scripts/BallBreakerGame.cs`; existing saves keep their previously stored state.

Booster	Unlock level	Initial grant	Effect / game-authored help
Zap	4	2	Tap Zap, then tap any numbered shape: it breaks at once.
Push Back	7	2	Sinks every shape 2 rows away from the line.
Ball Burst	11	2	Adds +6 balls to your next volley.
Quake	16	2	Halves the number on every shape in view.

To rebalance, edit the matching `BoostUnlock` and `BoostStart` values and the corresponding handlers; keep the four entries in the same order. To rename or translate, edit `BoostName` in `Scripts/BallBreakerGame.cs` and `BoostDesc` in `Scripts/BallBreakerGame.Hud.cs`. A targeted booster may require a second tap on an eligible object; read its handler and test invalid targets before changing consumption logic. Refills must be granted only after the

successful rewarded callback; cancelling an ad must not grant a reward.

Game text versus saved data and service configuration

Game progress, tutorial flags, currency and booster inventory are stored through the private `HzSave` helper and Unity PlayerPrefs. Search that helper and the main game script for the current key prefix. Changing displayed titles does not require changing those keys. Renaming keys can make an existing player appear to lose progress; plan migration before doing so.

Use a separate test profile when replaying onboarding. Do not use a global DeleteAll operation in a buyer's existing multi-asset project. Developer resets and unlocks should be used deliberately on test saves. Confirm the state after stopping/restarting Play, because an old saved tutorial flag can correctly suppress the popup you expect to see.

The HLK settings asset is `Assets/SatisVizionBallBreaker/HyperCasualLiveOpsKit/Resources/HLK/HLKSettings.asset`. Ads default to Dummy and IAP to Disabled. Keep those defaults while reskinning. A production provider requires its real, separately installed SDK, IDs, platform configuration and testing. Optional setup documentation does not mean a provider SDK is included or automatically installed. Keep service identifiers, ad-placement IDs and reward callback semantics separate from translated button text.

Reskin troubleshooting and release check

Symptom	Check
A label reverts after one frame	Edit the refresh-time `.text =` assignment listed above, not only the creation argument. Search the exact visible phrase in Scripts, including portrait and landscape branches.
A scene edit disappears after stopping Play	The HUD is generated at runtime. Save the source/data edit outside Play mode, then enter a fresh Play session.
A colour changes on the HUD but not a popup	Inspect that popup's colour arguments and the separate popup palette symbols; it may use a different surface/ink pair or a baked bitmap.
A translated string clips	Preserve interpolation/line breaks, shorten the caption or intentionally adjust its Text/RectTransform size. Test the narrowest supported portrait layout and any longer language.
The HLK language table does not change the game label	That label uses a literal or game data. Edit its actual source/data or implement an explicit localization lookup there.
New art or audio is missing	Compare the exact resource base name, folder, importer settings and code lookup. Preserve its .meta file and verify there are no compile errors.
The old tutorial still does not appear	Check persisted tutorial/booster-first-use flags with a disposable save. Reaching an already-completed tutorial in an existing profile is not a localization failure.
The browser/Android demo looks unchanged	Rebuild and redeploy that demo after source changes. A hosted manual update does not rebuild the game.
Production ads or purchases are unavailable	This delivery uses simulated ads and Disabled IAP. Install/configure a supported real provider separately and test the complete result.

Before your own release: review branding, UI contrast, tutorial wording, authored/generated levels, saved progress, every booster, pause/mute controls and both layouts. Set your own application identifier, icons, version, signing and release privacy/service settings. Perform actual target-device/browser testing; this manual and its documentation-only update do not certify your customized binary.

Licences, support and documentation access

Keep the included font, icon, TextMesh Pro, audio and HyperCasual LiveOps Kit notices with your distribution.

Marketing illustrations are separate from authentic gameplay screenshots. Refer to the included licence/AI disclosures for the supplied components; adding your own assets requires checking their distribution terms.

For product support, use the marketplace item support channel. SatisVizion support email: `satisvizion@gmail.com`. Include the game name, Unity version, target platform, exact Console error, changed files and clear reproduction steps. Do not send signing keys, passwords or private service credentials.

This is the main game manual. It is included offline in the main ZIP and the game's Documentation folder. Shared-kit detail is available in `HyperCasualLiveOpsKit.html` in the offline documentation bundle; within the Unity project, open the included kit's Documentation folder. The current online manual is linked in the publisher metadata. Installation and basic use do not depend on visiting an external storefront.